ee2004

Week5

Pointer-Based Operations

Pointers and pointer arithmetic allow reaching and modifying a specific memory address in C and C++ languages.

This is one of the most powerful and possibly confusing properties of C language. Since we are using pointers in the STM32F4 microcontroller, we can grasp the basic idea behind them more easily as we can observe the memory map of the microcontroller via STM32CubeIDE.

This will be of great help in understanding the usage of pointers and pointer arithmetic.

Next, we provide several usage areas of pointers in C language. We can revisit pointer-based operations in C++ language in Mbed Studio as well.

Pointer to a Variable

- We provide a pointer definition and usage in Listing 9.1. In this code, we define an integer variable a and assign value 3 to it. In fact, when we make such a definition and assignment, the variable is kept in a specific memory address. In order to reach this memory address, we define a pointer aPointer with integer type. This pointer can keep memory address of the variable a by the code line aPointer=&a.

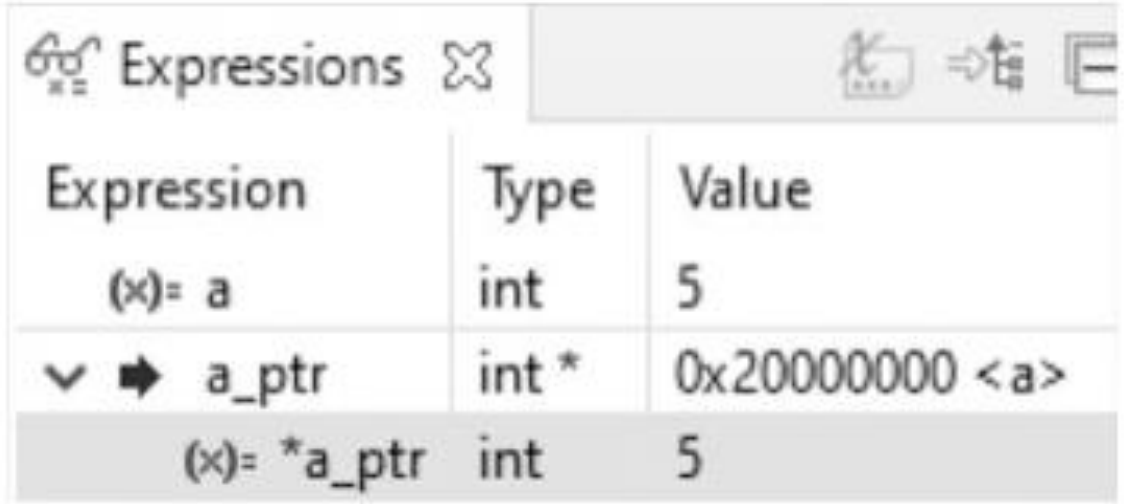
Listing 9.1 Pointer to a variable

```
int a = 3;
int *aPointer;

int main(void)
{
aPointer = &a;
*aPointer = 5;

while (1);
}
```

- For our case, this memory address is 0x20000000 as can be seen in the “Expressions” window in Fig. 9.2 after the code is executed in STM32CubeIDE. We can indirectly reach and modify this memory address content by the code line *aPointer=5. Hence, the a value is modified as can be seen in Fig. 9.2.



Pointer to a Pointer

- The pointer is itself another variable kept in memory. We can reach this memory address by using pointer to a pointer. This method is also called double pointer in literature. To note here, this is an advanced and powerful operation. We only use it for reaching the pointer address in this book.
- We provide the pointer to a pointer (double pointer) usage in Listing 9.2. In this code, we extend our previous example in Listing 9.1 such that we define the pointer to a pointer by the code line int **pPointer and equate it to the address of aPointer by the code line pPointer = &aPointer.

Listing 9.2 Pointer to pointer usage

```
int a = 3;
int *aPointer;
int **pPointer;

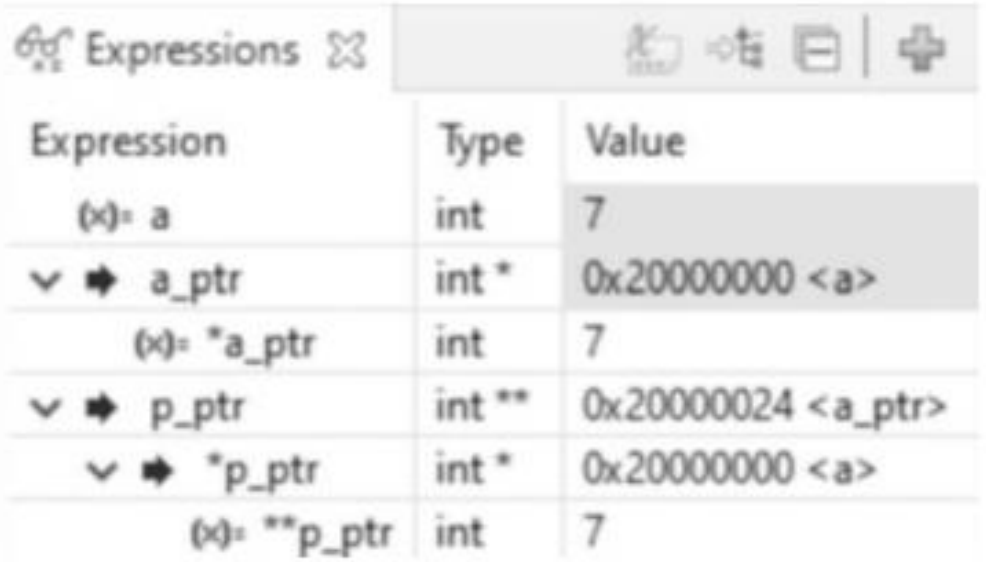
int main(void)
{
aPointer = &a;

pPointer = &aPointer;

*aPointer = 5;

**pPointer = 7;
```

- As we run the code in STM32CubeIDE, we can observe that the aPointer is kept in the memory address 0x20000024 as can be seen in the “Expressions” window in Fig. 9.3.
- In Listing 9.2, we modify the value of our variable a by the pointer to pointer usage as **pPointer=7.



Reaching a Specific Memory Address by Pointers

- Pointers can be used to reach a specific memory address in the STM32F4 microcontroller. We provide two methods for this purpose in this section. The first method is based on pointer arithmetic and casting operation. We provide such an example in Listing 9.3.

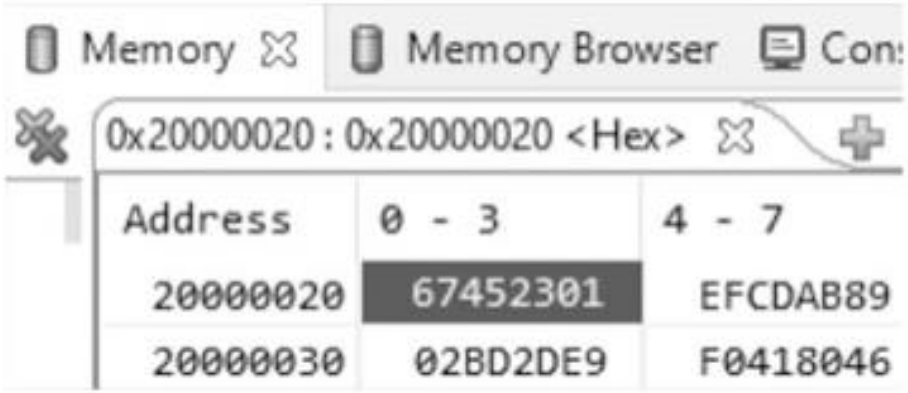
Listing 9.3 Reaching a specific memory address using pointer arithmetic and casting operation, the C code

```
unsigned int *memLoc;

int main()
{
memLoc = (unsigned int *)0x20000020;

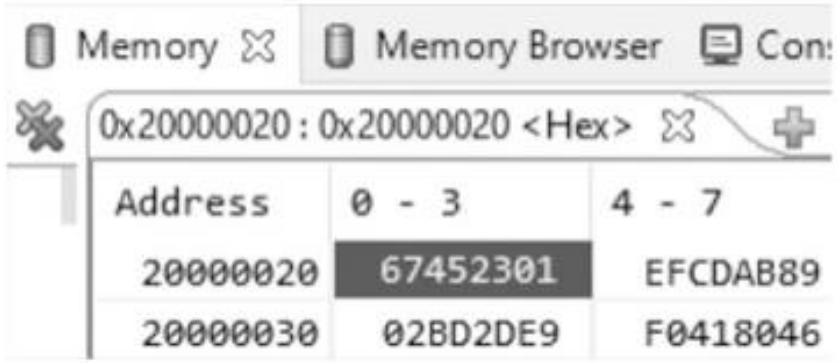
*(memLoc) = 0x01234567;
*(memLoc + 1) = 0x89ABCDEF;

while (1);
}
```



- In this code, we modify the memory addresses 0x20000020 and 0x20000024 via pointer arithmetic and casting operation. We provide the memory map after this operation in Fig. 9.4. As can be seen in this figure, the memory content has been modified as expected.

- The second method in reaching a memory address is using the define statement. We provide such an example in Listing 9.4. Here, we modify the memory addresses 0x20000020 and 0x20000024 via define statement. The memory map after this operation will again be as in Fig. 9.4.



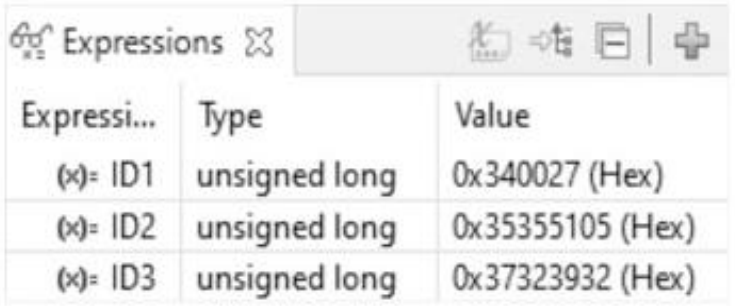
Listing 9.4 Reaching a specific memory address using the define statement, the C code

```
#define memLoc1 (*(unsigned long *)0x20000020)
#define memLoc2 (*(unsigned long *)0x20000024)

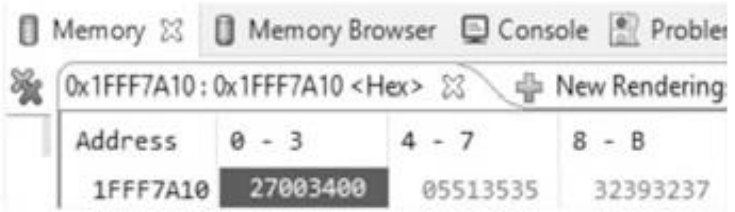
int main(void)
{

memLoc1=0x1234567;
memLoc2=0x89ABCDEF;

while (1);
}
```



(a)



(b)

- The second method allows obtaining the unique identification number (ID) of the STM324 microcontroller. As a side note, each STM microcontroller has its unique ID composed of 96 bits assigned to it. We provide the sample code for this operation in Listing 9.5. The obtained unique ID for our microcontroller can be observed both in the “Expressions” window and “Memory map” as in Fig. 9.5.

Listing 9.5 Obtaining unique ID of the STM32F4 microcontroller,

```
#define ID1 (*(unsigned long *)0x1FFF7A10)
#define ID2 (*(unsigned long *)0x1FFF7A14)
#define ID3 (*(unsigned long *)0x1FFF7A18)

int main(void)
{

while (1);
}
```

Pointers and Arrays

- When an array is defined in C language, it is in fact treated as pointer. We provide one such example in Listing 9.6. In this code, we first define an integer array, a, with entries {1, 2, 3, 4, 5}. We then assign the array to the pointer with the code line aPointer=a. We next reach the second element of the array and modify it by the code line *aPointer = 0. We can observe the effect of this operation in the “Expressions” window in Fig. 9.6. This window also provides another insight on the array usage. As can be seen there, the array a is kept by its starting address only. Hence, we can use it as a pointer.

Listing 9.6 Array usage as pointer example

```
int a[5] = {1, 2, 3, 4, 5};
int *aPointer;

int main(void)
{

aPointer = a;
aPointer += 1;
*aPointer = 0;

while (1);
}
```

Expressions

Expression	Type	Value
a	int [5]	0x20000000 <a>
(x)= a[0]	int	1
(x)= a[1]	int	0
(x)= a[2]	int	3
(x)= a[3]	int	4
(x)= a[4]	int	5
a_pointer	int *	0x20000004 <a+4>
(x)= *a_pointer	int	0

Pointer to a Structure

- Structure (struct) is a powerful tool in representing data in grouped form. We will be using structures in digital image processing operations. We can use pointers in reaching a specific field in a structure.

Listing 9.7 Structures and pointer usage on them,

```
#include <stdint.h>

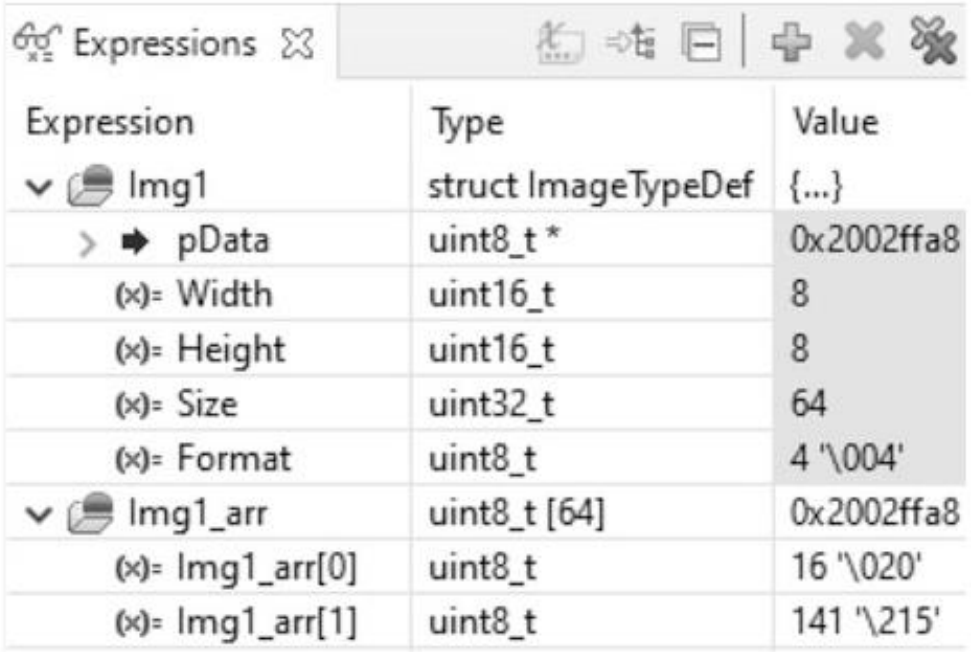
struct ImageTypeDef
{
uint8_t *pData;
uint16_t width;
uint16_t height;
uint32_t size;
uint8_t format;
};

int main(void)
{

struct ImageTypeDef img1;
uint8_t img1Arr[64];

img1.pData = img1Arr;
img1.width = 8;
img1.height = 8;
img1.size = 64;
img1.format = 4;
img1.pData[5] = 8;

while (1);
}
```



- We provide an example on structure usage in Listing 9.7. Here, we define an image struct, ImageTypeDef, with five fields as *pdata, width, height, size, and format. Then, we create an image struct as img1 and assign values to its elements. We also provide how a pointer is used to reach specific field elements in the structure in the same example. We provide the “Expressions” window in Fig. 9.7 as we run the code in STM32CubeIDE.

Function Call by Reference

- Pointers allow passing array addresses directly to a function. This is function call by reference which allows modifying a specific array element in the function. Here, we do not transfer all array elements to the function and receive them back afterward. This becomes extremely important while processing long arrays such as images or audio signals.

Listing 9.8 The first function call by reference example,

```
#include <stdint.h>

char a[5] = {1, 2, 3, 4, 5};
char *aPointer;

void replace(char *pointer);

int main()
{

aPointer = a;
replace(aPointer);

while (1);
}

void replace(char *pointer)
{
pointer += 1;
*pointer = 0;
}
```

- We provide two examples on the usage of function call by reference next. In the first example, given in Listing 9.8, the function replace is used to replace the second element of any array with value 0. Here, only starting address of the array is passed to the function instead of passing all its elements.

- In the second example, given in Listing 9.9, the function squareFunc is used to take square of the array elements. Here, only the starting address and size of the array are fed to the function instead of passing all array elements. Size of the array is fed to the function since it must know the number of array elements to operate on.

Listing 9.9 The second function call by reference example, the C code

```
char a[5] = {1, 2, 3, 4, 5};
char *aPointer;

void squareFunc(char *pointer, int size);

int main()
{
aPointer = a;
squareFunc(aPointer, sizeof(a));

while (1);
}

void squareFunc(char *pointer, int size)
{
int i;

for (i = 0; i < size; i++)
{
*(pointer + i) *= *(pointer + i);
}
}
```

Function Pointers

- Pointers can also be used to call functions. This is different from previous applications such that we call a code script to be executed by an assigned pointer to it. There are two main reasons for the usage of such a method. The first usage area is when a callback function is associated with its interrupt source in Mbed. A simple example for this usage is the code line button.rise(&toggle()). Here, toggle()) is the callback function associated with the button rise interrupt. The second usage area is forming a function which can perform the same operation on different variable types.

- We provide a function pointer usage example in Listing 9.10. This code consists of a function to modify array elements as desired. We feed start address of the array, size of the array, size of each array element, and the function (dummy) to perform the desired change as a pointer. The (dummy) function is called and operated on the given data. However, we do not know what this (dummy) function is until it is called. In our example, we set this (dummy) function as the one taking square of the array elements.

Listing 9.10 Function pointer usage example, the C code

```
/* USER CODE BEGIN PV */
float arr[5] = {1.7, 2.8, 3.6, 4.5, 5.2};
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void change(void *arr, size_t arrSize, size_t elem
    dummy)(const void *))
{
char *ptr = (char *)arr;

int i;
for (i = 0; i < arrSize; i++)
{
dummy(ptr + i * elemSize);
}
}

void squareFunc(const void *x)
{
*(float *)x = (*(float *)x) * (*(float *)x);
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
change(arr, 5, sizeof(float), squareFunc);
/* USER CODE END 2 */
```

Local, Global, and Static Variables

- We can define a variable either as local or global in C and C++ languages. As the name implies, a global variable is available to all code blocks. However, a local variable is only available to the function it is defined in. These two variables are kept in different memory locations in RAM. Global variables are kept starting from the lowest possible memory address (0x20000000) in SRAM. As a new global variable is added, the memory address is incremented and the new variable is saved there. On the other hand, local variables are kept in the highest memory address in the stack. For STM32F4 microcontroller, this address is initially 0x2002FFFF which is the end of SRAM. Based on the definition of stack, when a new local variable comes, it is saved to the recent memory address. The address value is decremented afterward. Hence, local variables are saved from top to bottom in SRAM in stack. It is important to note that the C and C++ languages take main() as a function. Hence, a variable defined within the main function is also treated as local. Therefore, a global variable should be defined before the main() declaration.

- We should also explain how a static variable works in C and C++ languages. Such a variable is created with the static keyword and kept in RAM during code execution. Even if a static variable is declared as local, it is not kept in stack. Instead, the static variable is placed at the beginning of RAM as with global variables. The importance of the static variable is as follows. It can be defined in a function. Hence, it will be local to that function. However, whenever the function is called more than once, the value of the static variable is not lost unlike other local variables.

- We can observe how global and local variables are stored in memory by the help of pointer usage. We provide one simple example in C language in Listing 9.11. Here, the variable a is defined as global and the variable b is defined as local. We provide the “Expressions” window after executing the code in Fig. 9.8. As can be seen in this window, the two variables are kept in different memory locations. We can observe this effect by pointer usage.

Listing 9.11 Local and global variables, the C code

```
//Global variable
int a = 1;
int *aPointer;

int main(void)
{
//Local variable
int b = 2;
int *bPointer;

aPointer = &a;
bPointer = &b;

while (1);
}
```

